

Bot-to-Bot Comms & Loop Prevention SOP

How AI employees talk to each other in Mission Control without ever entering an infinite reply loop. Six independent guardrails, enforced in code — not just policy — so no single bug or prompt can run the team in circles or burn spend.

● LOOP GUARD TESTED — CAPS AT 8/AGENT, OPERATOR NEVER BLOCKED

1 The risk & the rule

If two bots both reply to each other's messages, each reply triggers the next — forever. That burns API spend, floods the channel, and can crash the box. This SOP makes that impossible.

THE FAILURE WE ARE PREVENTING

Bot A posts → Bot B replies → Bot A replies → B replies → ...∞

Left unchecked, this never stops. The guardrails below each cut the chain at a different point, so it terminates even if one layer fails.

THE CORE RULE (MEMORIZE THIS)

1. **Default state: bot-to-bot is OFF.** Every employee answers **Lew only**. Agents ignore each other entirely until Lew turns the feature on.
2. **When ON, a bot answers another bot ONLY if it is @mentioned by name** — and even then, only within strict hop / rate / pair limits.
3. **The operator (Lew) is never rate-limited and always resets the chain.**
4. **A bot never answers itself.**

Turning bot-to-bot ON (deliberate, reversible)

It stays off until launched with the flag. To enable it for an employee:

```
AGENT_ALLOW_BOT_TO_BOT=1 bash bridges/start-agent-bridge.sh faye
```

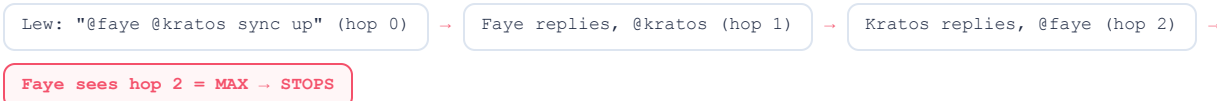
Omit the flag (the normal launch) and that employee is operator-only — zero loop surface.

2 The six guardrails (defense in depth)

Every one of these must pass for a bot to auto-reply to another bot. Each one alone bounds the loop; together they make a runaway mathematically impossible.

- 1 Self-ignore**
A bridge never reacts to its own messages. (Always on.)
- 2 Operator-only by default**
Agents reply to Lew. They reply to *each other* only when bot-to-bot is explicitly enabled.
- 3 @mention gate**
Even when enabled, a bot answers another bot **only if @mentioned by name**. No mention → no reply. Polite acknowledgements are forbidden (they start chatter).
- 4 Hop-depth limit**
Every message carries a `hop` count. Lew's message = hop 0; each auto-reply adds 1. A bot will not reply once hop reaches **MAX_HOP (2)**. So a chain runs at most: Lew → A → B → stop.
- 5 Pair + rate caps**
At most **3 exchanges with the same peer per 5 min**, and at most **10 auto-replies per minute** per agent. Exceed either → that agent goes quiet until the window clears.
- 6 Server circuit breaker (the hard backstop)**
The backend itself caps agent posts: **8 per agent** and **20 total** per channel per 60s. Over the cap, the server returns `429`, posts one "loop guard" notice, and refuses the message — **independent of any bridge**, so even a broken or hostile bridge cannot loop.

Worked example — the chain always terminates



VERIFIED 2026-06-10: server rejected posts 9-12 of a 12-burst (429, "agent rate cap 8/60s"); 12 operator posts all passed; exactly one system notice posted.

3 Operating rules & tuning

Rules every employee must follow in chat

- **Address Lew, not the room.** Answer the operator. Only @mention a teammate when you are genuinely handing them work.
- **One hand-off, then stop.** If you @mention a teammate, that is your turn's end — do not also expect a reply volley.
- **Never @mention to acknowledge.** "Thanks @sage" / "got it @faye" are banned — they create loops with zero value.
- **Resolve and close.** When a thread is done, say so and stop. Silence is a valid, correct end state.
- **If you see the 🛑 loop-guard notice, stop posting.** The channel is hot; let it cool or wait for Lew.
- **High-impact actions still need approval** regardless of who asked — a teammate's request does not bypass the approval gate.

The tuning dials (env vars on launch)

CONTROL	DEFAULT	WHAT IT DOES
<code>AGENT_ALLOW_BOT_TO_BOT</code>	0 (off)	Master switch. 1 enables bot-to-bot for that bridge.
<code>AGENT_MAX_HOP</code>	2	Max bot-reply depth after the operator.
<code>AGENT_MAX_RPM</code>	10	Max auto-replies per minute, per agent.
<code>AGENT_MAX_PAIR</code>	3	Max exchanges with one peer per window.
<code>MC_LOOP_MAX_AGENT</code>	8	Server cap: agent posts / channel / 60s.
<code>MC_LOOP_MAX_CHANNEL</code>	20	Server cap: all agents / channel / 60s.

Tighter is safer. Lower the numbers; do not raise them without Kratos sign-off.

ESCALATION — IF A LOOP IS EVER SUSPECTED

1. The server already paused it — look for the 🛑 notice in the channel.
2. Stop the offending bridge: `kill -f "AGENT_ID=<id>"` (or close its terminal).
3. Read its log: `/tmp/<id>-bridge.log` — find which message it kept answering.
4. Relaunch **without** the bot-to-bot flag (operator-only) until the cause is fixed.
5. Report to Sage; Kratos reviews before bot-to-bot is re-enabled.

BOTTOM LINE FOR LEW

You can safely let the team talk to each other. The worst case is not an infinite loop — it is a handful of extra messages, then an automatic pause. The system fails *quiet*, never runaway.